

KK Modding 101

Resumen épico de Wogrim sobre cómo hacer todo tipo de mods

Terminología e información básica

Los **mods** son cualquier cosa que añade un "nuevo elemento" (término utilizado de forma muy imprecisa) al juego. Son tipos de cosas que ya existen en el juego y que la gente quiere más para tener más variedad, o la gente quiere crear algo específico pero no tiene la opción perfecta. Puede ser un elemento de textura muy simple, como una nueva opción de nariz, o algún tentáculo animado en 3D.

Un **port** es un objeto que es una copia de un objeto de otro juego. Como modder esto significa que parte del trabajo se ha hecho por ti porque no estás haciendo el artículo desde cero, pero a menudo tienes que hacer algunas modificaciones para que funcione correctamente. Los ports de ropa siguen dando mucho trabajo por razones que se explicarán más adelante.

Los **plugins** son programas que cambian el funcionamiento del juego; a efectos de esta guía no los llamaremos mods. Si un plugin crea un nuevo tipo de elemento para el juego, lo llamaremos mod si hace un elemento de este tipo, como un nuevo cuerpo para el plugin Uncensor Selector. Tales mods generalmente necesitan un componente extra para que el plugin los lea. Los plugins son increíbles y nos dan más opciones de lo que podemos hacer con los mods. Todos los plugins que menciono en esta guía vienen con HF Patch, así que si lo tienes, estás listo (y si tu mod depende de uno de estos plugins, es de esperar que tus usuarios ya los tengan).

Los **Hard Mods** son mods que cambian directamente los datos del juego, lo que suele considerarse una mala idea porque es fácil romper la instalación del juego y, en cierto modo, no puedes compartirlos con otras personas.

Los **Sideloader Mods** son mods que el plugin Sideloadler lee de la carpeta "**mods**" en la carpeta de instalación del juego. Suelen guardarse con una extensión de archivo **.zipmod**, por lo que también se llaman Zipmods. Todos los mods que hagas deben ser de este tipo. Tu zipmod no funcionará si la estructura del archivo no es correcta.

Unity es el motor de juego con el que funciona KK. Su descarga es gratuita, pero si quieres usarlo para hacer mods en KK necesitas la **versión 5.6.2f1**. Puedes hacer muchos tipos de mods sin él. Hay un proyecto de Unity llamado **Koikatsu Modding Tools** que hace algunas cosas extra para ayudar a armar un mod, pero si no entiendes la estructura que debe tener tu mod será difícil solucionar cualquier problema que tengas.

Asset Bundle (abreviado como **AB**) es el nombre de un tipo de archivo del Unity game que tiene la extensión de archivo **.unity3d**. Casi todos los datos de los objetos del juego están en ABs, y casi todos los datos de la mayoría de los tipos de mods van en ABs. Así que la mayoría de los zipmods tienen al menos un AB. Los ABs tienen un **CAB String** que debe ser diferente de todos los demás en el juego y en otros mods.

SB3UGS (también llamado SB3U, **SB3Utility**, y probablemente otros nombres) es un software que le permite ver y modificar ABs. Para hacer un mod, puede hacer que Unity construya un nuevo AB con todos los datos de sus artículos, o puede copiar un AB existente y usar SB3UGS para modificarlo. Incluso si construyes tu AB con Unity, SB3UGS es MUY útil para mirar los objetos del juego y otros mods para aprender cómo funcionan, y para solucionar los problemas de tu mod si no funciona.

El **Manifest** es un archivo necesario en un zipmod que contiene metadatos sobre su zipmod. Debe llamarse **manifest.xml** y debe estar en la raíz de su zipmod y debe tener un **GUID** que no entre en conflicto con otros mods.

El **List File** es un archivo en un zipmod con formato **.csv** que le dice al juego qué objetos hay en el mod. El juego tiene archivos de lista para sus propios artículos, pero están dentro de los AB. Los archivos de lista tienen diferentes formatos para cada tipo de ítem, así que un mod con múltiples tipos de ítems tendrá múltiples archivos de lista. El juego se confundirá y parecerá roto si un archivo de lista tiene un formato incorrecto, así que compruébalo cuidadosamente cuando hagas un mod. Puedes hacer un archivo de lista con un editor de texto normal, pero te recomiendo un editor de tablas para asegurarte de que las columnas se alinean correctamente.

Maker es un nombre comúnmente utilizado para el sistema de creación de personajes del juego, que es donde se va a probar inicialmente muchos tipos de mods. El **Saturation Filter** es un efecto que se aplica a la cámara y que generalmente oscurece y sobresatura los colores, por lo que tus objetos pueden tener un aspecto diferente al esperado. Hay un plugin que puede desactivarlo, pero se recomienda dejarlo activado y colorear tus objetos en consecuencia.

Free H es otro nombre para el modo Tener Sexo, que es donde se probarían tipos de mods menos comunes, como una nueva posición sexual o un nuevo mapa. **Free H** tiene diferentes filtros de saturación para el Día, el Atardecer y la Noche, así que puedes considerar cómo se ven tus objetos con ellos.

El **Adventure Mode** (también llamado **Story Mode**) es el juego principal en el que interpretas a un chico que es transferido a una escuela de chicas. Algunos tipos de objetos se supone que tienen una versión de bajo detalle para el rendimiento del juego (llamada versión **_low**) cuando se camina por la escuela (**Roaming**). Pero hay un plugin que mucha gente usa llamado **Force High Poly** que significa que la versión de bajo detalle no se usa. Puedes mirar en los archivos del juego para ese tipo de ítem para ver si se supone que tienes una versión **_low**.

Studio (o **Chara-Studio**) es el sistema en el que se configuran las escenas con personajes y objetos para hacer buenas capturas de pantalla o hacer una película, que se lanza como un programa separado del resto del juego. Algunos objetos pueden hacerse **posables** en Studio con los controles de **Kinematics (FK)**. También hay un plugin llamado **KKPE** que te permite posar cosas que no puedes posar con FK, pero lo encuentro menos fácil de usar.

Un **Shader** es un código que le dice a tu tarjeta gráfica cómo dibujar un elemento (dadas las texturas y otras variables). Un shader va en el **Material** de un ítem, pero el material básicamente no hace nada excepto mantener el shader, por lo que estos términos a menudo se refieren a la misma cosa. En general, sin embargo, la palabra Material se utiliza cuando se habla de un elemento específico. Hay varios shaders en el juego para diferentes tipos de ítems para soportar diferentes características. Puedes cambiar la mayoría de las texturas y variables del material de un objeto en Maker o Studio con un plugin llamado **Material Editor** (abreviado como **ME**) para ver cómo funcionan los shaders y decidir qué ajustes quieres para tu objeto.

Un **MonoBehaviour** (abreviado como **MB**) es un script en Unity que puede ser puesto en ítems. Cada tipo de ítem en 3D necesita un MB correspondiente a ese tipo de ítem; el juego necesita el MB para la información de mostrar su ítem. También puedes poner **MBs personalizados** en un ítem, para darle alguna otra funcionalidad.

Los **Layers** son una característica de Unity que permite que las cosas en el juego existan básicamente en diferentes dimensiones, que pueden ser configuradas para interactuar o no interactuar entre sí de varias maneras. En el caso de KK, los personajes (y los objetos unidos a ellos) están en el **Layer Chara (A en SB3U)** y el mundo está en el **Layer Map (B en SB3U)**. Esta separación significa que la iluminación del personaje (luz direccional que normalmente apunta en la misma dirección de la cámara) no ilumina el mundo, y cualquier luz del mapa (como las colocadas en Studio) no ilumina al personaje. Así que cualquier elemento 3D que hagas debe estar en el Layer correcto.

Los tipos de Main Item

Los **Texture Items** incluyen varios tipos de elementos como pezones, líneas de bronceado, ojos, nariz, lápiz de labios. Estos son bastante fáciles de hacer si sólo echas un vistazo a cómo funcionan los ítems del juego, pero pueden necesitar un poco de esfuerzo para conseguir el aspecto exacto que deseas. Técnicamente se pone en un elemento 3D, pero no tienes control sobre esa parte.

Los **Accessories** se fijan al personaje en un lugar elegido para que puedan moverse con él. Los accesorios simples son bastante fáciles de hacer, los animados son difíciles. Hay un poco de formato requerido para la estructura del artículo para que pueda ser movido y redimensionado con los controles del juego. Los accesorios deben tener el **MB ChaAccessoryComponent**.

Los **Studio Items** son objetos exclusivos de Studio que los jugadores pueden colocar en sus escenas. Los ítems de Studio tienen un sistema de list file desordenado para poner los ítems en una carpeta específica. Hay un plugin llamado **QuickAccessBox** que da una opción de búsqueda, y te permite dar a los elementos de Studio un icono en miniatura (que no tienen en la interfaz normal). Hay un formato especial para este icono y no se especifica en el archivo de la lista como para la mayoría de los elementos. Los objetos de Studio son similares a los accesorios en el sentido de que puedes hacerlos simples o difíciles. Los objetos de Studio generalmente deben tener el **MB ItemComponent**.

El **Hair** es una categoría muy importante que requiere algo de trabajo para hacerlo correctamente (las razones son posteriores). Si se hace como un accesorio se puede mover alrededor, lo cual es genial para cosas como picos de pelo y colas de caballo laterales al azar. El pelo debe tener el **MB ChaCustomHairComponent**. Para que un accesorio sea tratado como pelo necesitas el plugin **KK_HairAccessoryCustomizer** y debe tener tanto el MB de accesorio como el de pelo.

Clothes es lo que más trabajo da para hacerla bien. Los zapatos y los calcetines son de dificultad media, todo lo demás es una búsqueda épica. Sin embargo, hay algunos atajos que puedes tomar, y puedes decidir ignorar las características de la ropa que no te interesan. La ropa normalmente debe tener el componente **ChaClothesComponent**. La ropa se puede convertir en una especie de accesorio con el plugin **KK_AccessoryClothes** y el **MB ChaAccessoryClothes**, que permite que un accesorio se mueva con el cuerpo como la ropa (en lugar de estar unido sólo en un punto), pero en todos los demás aspectos se trata como un accesorio, no como ropa.

Mods de texturas

Los **Texture Mods** son mods que sólo tienen nuevas texturas, que en su mayoría son fáciles de hacer. Pueden ser elementos de textura, o cambios en las texturas de cualquier elemento 3D.

Para hacer un texture mod para un ítem 3D, sólo tienes que cambiar las texturas (y otras variables del shader si quieres). El **UV Mapping** (a menudo llamado simplemente **UVs**) es lo que define qué parte de la textura va a qué parte del elemento; esta información es útil para decidir dónde dibujar cosas. También hay software que te permite hacer **Texture Painting**, que es cuando dibujas directamente en el elemento 3D. Mucha de la ropa interior y los sujetadores del juego son en realidad sólo cambios de textura en los mismos modelos de pantalones cortos y camisas ajustados a la piel; puedes hacer los tuyos propios de la misma manera. La mayoría de los shaders del juego para artículos 3D tienen las siguientes ranuras de textura disponibles (pero no todas tienen que ser utilizadas):

- **MainTex** es lo que define el color base de un elemento si no utiliza el selector de color, de lo contrario debe ser blanco. La transparencia se puede utilizar para ocultar partes del objeto. En los objetos del juego, el nombre suele terminar en **_t**
- **DetailMask** te permite pintar luces, sombras y líneas en un elemento, con las sombras y líneas reaccionando a la configuración del juego de sombras y contornos. En los objetos del juego, el nombre suele terminar en **_md**
- **LineMask** da básicamente las mismas opciones que DetailMask pero funcionan de forma ligeramente diferente, y no se utiliza en muchos elementos. En el juego, el nombre suele terminar en **_line**
- **ColorMask** te permite establecer áreas de un ítem como coloreables con el selector de color con canales rojo, verde y azul. En los elementos del juego, el nombre suele terminar en **_mc**
- El **NormalMap** cambia el ángulo con el que se calcula que la luz incide sobre el elemento, lo que puede utilizarse para que los pequeños detalles tengan un aspecto más 3D. Normalmente se calcula por software, pero los shaders del juego lo necesitan en un formato especial. En los objetos del juego, el nombre suele terminar en **_n**
- **AnotherRamp** se ve generalmente en los artículos que tienen una parte que se supone que es de metal; interactúa con el brillo de la textura para dar menos brillo en diferentes ángulos por lo que brilla cuando se mueve (aspecto reflectante falso). No utiliza los UVs como otras texturas. En los objetos del juego, el nombre suele empezar por **Ramp**

Unskinned Item Mods

Un **Modelo 3D** o **Malla** es la estructura física (normalmente hecha de muchos triángulos) que generalmente define la forma de un elemento (los shaders pueden cambiar la forma). Llamamos a la malla **Unskinned** si no se mueve en relación a sí misma. Así que un ítem como una espada es un accesorio unskinned que se mueve cuando el personaje se mueve, pero la espada en sí no se dobla ni nada. Muchos accesorios y objetos de studio son unskinned.

El **software de modelado** como **Blender** (gratuito) o **Maya** (estándar de la industria) se utiliza para hacer estos artículos; pueden tomar algunas horas para aprender, pero una vez que usted tiene experiencia es sólo dos pasos adicionales de la creación de un mod de textura para un elemento 3D: un paso es la creación de la malla, y el otro se llama **UV Unwrapping** que es cómo se define el mapeo UV. Si necesitas aprender a usar el software de modelado, hay muchos videotutoriales en YouTube.

Un port de un elemento unskinned es relativamente fácil, pero espera hacer cambios de textura. Por lo general, querrá cambiar el MainTex y crear un ColorMask para que sea coloreable, puede crear un DetailMask si quiere características relacionadas, y probablemente tendrá que convertir el NormalMap si tenía uno.

Skinned Item Mods

Una malla **Skinned** es aquella que está unida a un **Armature** (también llamada **esqueleto**) de modo que cuando los huesos se mueven, las partes correspondientes del elemento se mueven con él. El cuerpo del personaje es una malla Skinned porque las partes se mueven según las animaciones regulares, las animaciones dinámicas (como el movimiento de las tetas), y cambia de forma con los deslizadores en el Maker. Un item debe estar skinned para cualquiera de estos movimientos parciales. El **rigging** es el proceso de construir el esqueleto, y el **skinning** es el proceso de unir la malla al esqueleto (ambos se hacen en el software de modelado). Esto se puede hacer antes o después de hacer las texturas del objeto.

Hay algunas técnicas para el skinning. La forma manual es "pintar a mano" (llamado **Weight Painting**) cuánto afecta cada parte de la malla a cada hueso (llamado **Bone Weight**). Esto es bastante fácil para las cosas robóticas donde cada pieza es sólo afectada por un hueso, pero puede tomar muchas horas para las partes afectadas por múltiples huesos (**el máximo de Unity es 4**). Hay un cálculo de pesos Automático para huesos que puede obtener resultados bastante buenos si usted está haciendo su propio esqueleto, pero no es muy bueno para la ropa. El método ideal para la ropa es **copiar los Bone Weights** de un elemento ya existente de forma similar o del propio cuerpo del personaje, para que el elemento se mueva bien con la piel del personaje. Es probable que aún veas algo de **clipping** en las animaciones que necesita ser arreglado con el Weight Painting; lleva la malla/esqueleto del cuerpo y algunas animaciones a tu software de modelado para una prueba rápida.

El **Hair** (y los accesorios que funcionan como pelo) suelen tener skinned para que no sean rígidos como una roca. Para conseguir que el pelo se mueva, el objeto debe tener un MB especial llamado **DynamicBone** que hace los cálculos de física en los huesos que le indiques. Todo el pelo debe ser mapeado con UV de una manera particular para trabajar con el sistema de brillo del pelo, y el color mask debe ser configurado para que la gente pueda cambiar los colores de la raíz y la punta del pelo; mira algunos pelos del juego para ver ejemplos. Para colocar correctamente el **non-accessory hair**, tienes que hacer una reubicación especial de la cabeza antes de llevarla a tu software de modelado para asegurarte de que el pelo se alinea con él. El non-accessory hair puede ser colocado con FK si utilizas nombres de huesos específicos que los pelos del juego utilizan.

Los **Clothes items** están unidos al esqueleto del personaje, por lo que normalmente no hay que hacer ningún tipo de rigging para ellos. Pero hay un plugin llamado **ModBoneImplantor** que te permite añadir huesos si quieres algún tipo de prenda colgante que se mueva por ahí con huesos extra que tú mismo riggeas. Esto requiere que un MB especial llamado **BoneImplantProcess** sea puesto en tu item, además de DynamicBone para darle física. Estos huesos añadidos no se pueden posar con FK en Studio.

Los **Half States** (también llamados **Shift y Hang States**) son una característica exclusiva de algunas ranuras de ropa. Se trata de una malla separada que se muestra al desvestir a un personaje. Las mallas deben colocarse en lugares específicos dentro del item para que el juego pueda encontrarlas y desactivarlas correctamente. Los Half states suelen compartir las mismas texturas que el Full state, pero tienen una malla diferente. Para crear un Half state para tu propio ítem, es el menor trabajo si tienes el Full state 100% completado primero (incluyendo el skinning) y luego lo copias y editas para un Half state.

Algunas ranuras de la ropa y el cuerpo reciben una textura **AlphaMask** del list file del item en la capa por encima de ellos, que oculta partes dependiendo del state del otro elemento para evitar algún clipping. Así que la ropa que recibe un AlphaMask necesita los **UVs unwrapped** como la ropa del juego para que esto funcione correctamente. Estas máscaras en los list files se llaman **OverBraMask**, **OverInnerMask**, y **OverBodyMask** dependiendo de la capa que ocultan. Las ranuras de ropa apropiadas también deberían tener texturas para **Líquidos** (semen, cosas sexuales), pero puedes usar las del juego por defecto si los UVs son similares a los items del juego.

Los **Studio Items** a veces son skinned para que puedan ser posados con FK. Un objeto de este tipo necesita un list file extra llamado **ItemBoneList** para decirle al juego en qué huesos permitir el FK.

Otros tipos de mods

Un **Clothes Item Slot Move** es un mod que pone un ítem de ropa existente en una ranura diferente, para que pueda ser usado al mismo tiempo que otro ítem en su ranura anterior (no temas, también está disponible en su ranura anterior). Esto no se ve muy bien si los ítems se colisionan mucho, no sirve de nada si tu personaje ya está usando todas las ranuras de ropa, y hay un problema con los half states de la ropa que está fuera del alcance de esta guía. PERO en los casos en los que puedes hacerlo no hay un mod más fácil de hacer porque la entrada de tu list file sólo apunta a un ítem ya existente, por lo que tu mod ni siquiera necesita un AB. Hay una manera de arreglar algunos casos del problema de los half states de la ropa, pero requerirá que hagas una copia del ítem existente y hagas pequeñas modificaciones.

Un **Clothes Item Merge** es un mod que fusiona dos ítems de ropa juntos, lo que se haría por razones similares a un Clothes Item Slot Move. Esto se puede hacer a veces con un poco de trabajo en el list file y AB para decirle al juego que dibuje las mallas de ambos artículos al mismo tiempo, pero usted está limitado a 2 texturas de color elegido, y algunos artículos ya utilizan ambos. Puedes evitar este problema combinando las mallas y las texturas en una sola pieza, pero los UVs tendrán que ser movidos para compensar, así que es un poco molesto. Y no se puede colorear por separado. Y no se puede tener half states / ocultar los elementos por separado lo que puede romper su inmersión de waifu-undressing.

Un **Self-Animated item** es un ítem skinned que tiene una animación incorporada. Estas son cosas como alas o colas que se mueven en un patrón fijo, o tal vez sólo algo que gira constantemente. Usted usa Unity para asegurarse de que la animación esté incorporada, pero puede crear la animación misma en un software de modelado.

Un **Studio Filter** (también llamado **Scene Effect**) utiliza una imagen especial llamada LUT para hacer una corrección de color rápida (pero de baja calidad) a lo que la cámara ve; este es el mismo método utilizado por el Filtro de Saturación mencionado anteriormente. Es fácil de hacer una vez que se entiende cómo hacer una LUT. KK Modding Tools tiene un ejemplo.

También se pueden crear **Character Poses** y **H-Scene Animations** (posiciones sexuales). No sé los detalles de cómo crearlas, pero probablemente son más fáciles que los ítem skinned una vez que se supera la curva de aprendizaje y se tienen los IKs configurados para mover los cuerpos de los personajes. Omega tiene una guía sobre esto, que también contiene un proyecto de Blender ya configurado para facilitar la animación.

Un **Map Mod** añade un nuevo mapa al juego, para usarlo en Studio o en Free H. Es más trabajo configurar las ubicaciones de los sex locations (HPoints) pero KK Modding Tools tiene un ejemplo para mirar. Los Map Mod necesitan ser creados en Unity.

Un **Body Mod** es cuando se crea un nuevo cuerpo para ser seleccionado con el plugin Uncensor Selector. No sé cómo se hace (aparte de ajustar el cuerpo del juego), pero puedo decir algunas razones por las que generalmente no hacen grandes cambios en el cuerpo:

- Las animaciones del juego se estropearán si cambias el rigging
- La ropa dejará de encajar si se realizan cambios significativos en la malla
- Las formas en que la gente querría cambiar el cuerpo son más fáciles como mods de textura y accesorios

Un **Head Mod** tiene más o menos los mismos problemas que un body mod, pero en lugar de problemas de ropa son partes de la cara. Hay un par de buenos que cambian los dientes y uno que quita las orejas humanas para que puedas poner cómodamente un accessory ears. KK Modding Tools viene con un ejemplo para mirar.

Un **Shader Mod** es un mod que pone un nuevo shader en el juego para su uso en ME; la información sobre su shader se pone en una parte del manifest con un formato especial. Esto permite a la gente cambiar un elemento a su shader y cambiar sus propiedades con ME. Además, otros modders pueden copiar el shader para crear artículos con tu shader ya en él. Es fácil convertir un shader existente en un Shader Mod, pero consume más tiempo si quieres aprender a crear tu propio sombreador, y consume MUCHO tiempo si quieres crear un shader que funcione con características importantes del juego como el selector de color, o las sombras y el outline que reaccionan a los ajustes gráficos del juego.

Traducido por :

deadshark ° • (>~<) • °